

Full Stack Zero to Hero

CHAPTER 01

HTML Fundamentals

Chapter Notes and Reference Guide

Full Stack Web Development

| Module 1 of 12

| Beginner Level

Topics Covered: 01 What is HTML | 02 Document Structure | 03 Elements and Attributes | 04 Headings | 05 Text Formatting | 06 Links | 07 Images | 08 Lists | 09 Block vs Inline | 10 Comments | 11 Best Practices

Table of Contents

01	What is HTML?	Definition and purpose, browser rendering pipeline, HTML/CSS/JS relationship diagram, brief history and HTML5
02	HTML Document Structure	DOCTYPE declaration, html root element, head section elements, body section, full boilerplate with comments, document tree diagram
03	HTML Elements, Tags and Attributes	Element anatomy diagram, void/self-closing elements, attribute syntax, global attributes table, nesting rules with examples
04	Headings	h1 through h6 hierarchy diagram, SEO importance, accessibility and screen readers, heading usage reference table
05	Paragraphs and Text Formatting	p, br, hr tags; semantic vs visual formatting; strong vs b, em vs i; mark, del, ins, sup, sub, pre -- full comparison table
06	Links (Anchor Tags)	Anchor tag syntax, absolute vs relative URL table, target blank and security with noopener noreferrer, special link types, path diagram
07	Images	img tag with src and alt, attribute reference, alt text best practices, image format comparison table (JPG/PNG/GIF/SVG/WebP), responsive images
08	Lists	Unordered list, ordered list with type attributes, definition list, nested lists with code examples, list type selection table
09	Block vs Inline Elements	Block vs inline behavior explained, layout diagram, block elements table, inline elements table, div and span comparison
10	HTML Comments	Comment syntax, single and multi-line examples, when to use, when NOT to use, security warning
11	HTML Best Practices	Comprehensive checklist table covering DOCTYPE, lang, charset, lowercase tags, closing tags, alt text, indentation, validation

Topic 01 -- What is HTML?

Definition and Purpose

HTML stands for HyperText Markup Language. It is the standard language used to create and structure content on the web. HTML uses a system of tags (markup) to define different parts of a document -- headings, paragraphs, links, images, and more. HTML is **not** a programming language. It has no logic or loops. It is a **markup language**: it describes the structure and meaning of content, telling the browser what each piece of content IS, not how it looks.

How Browsers Parse and Render HTML

When you navigate to a webpage, the browser performs these steps in order:

1. DOWNLOAD -- Browser requests the HTML file from the server over HTTP/HTTPS.
2. PARSE -- Browser reads HTML top-to-bottom, building a tree structure called the DOM.
3. BUILD DOM -- Each HTML element becomes a node in the Document Object Model tree.
4. FETCH ASSETS-- Browser requests linked CSS, JavaScript, and image files.
5. RENDER TREE -- HTML structure is combined with CSS styles to create the render tree.
6. PAINT -- Browser paints pixels to the screen based on the render tree.
7. EXECUTE JS -- JavaScript runs and can modify the DOM dynamically.

The Relationship Between HTML, CSS, and JavaScript

A modern web page is built with three core technologies, each with a distinct role:

The Three Layers of a Modern Web Page

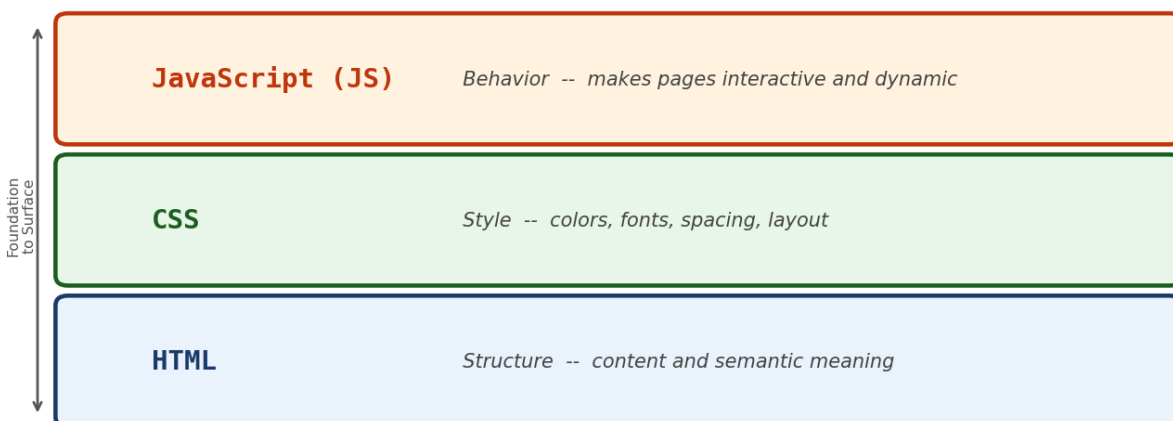


Figure 1.1 -- Three technology layers of a web page

Layer	Technology	Role	Example
1 - Foundation	HTML	Structure and content definition	<code><h1>, <p>, , <nav></code>
2 - Style	CSS	Visual presentation and layout	<code>color: blue; font-size: 18px;</code>
3 - Behavior	JavaScript	Interactivity and dynamic updates	<code>document.getElementById("btn")</code>

Brief History and HTML5

HTML was created by Tim Berners-Lee in 1991 at CERN. It has evolved through many versions. The current standard is HTML5, maintained as a living standard by the WHATWG (Web Hypertext Application Technology Working Group).

Year	Version	Key Milestone
1991	HTML 1.0	Created by Tim Berners-Lee at CERN. Basic text and links.
1995	HTML 2.0	First formal standard. Tables, forms, images.
1997	HTML 4.0	CSS support, frames, scripting. Major improvements.
1999	HTML 4.01	Refined standard. Widely used for many years.
2014	HTML5	Current major version. Semantic elements, video/audio, canvas, APIs. Backed by W3C and WHATWG.
Ongoing	HTML Living Standard	HTML is continuously updated by WHATWG -- no more numbered versions.

Topic 02 -- HTML Document Structure

DOCTYPE Declaration

The DOCTYPE declaration is the very first line of every HTML file. It tells the browser which version of HTML is being used. Without it, browsers may enter "Quirks Mode" and render the page inconsistently.

```
<!DOCTYPE html>
```

For HTML5 this is simply `<!DOCTYPE html>` -- short, lowercase, easy to remember. This is all you need.

The <html> Root Element and lang Attribute

The `<html>` element wraps ALL page content. The `lang` attribute specifies the document language for screen readers, browsers, and search engines.

```
<html lang="en">

</html>
```

The <head> Section -- Metadata Elements

The `<head>` section contains metadata about the page (not displayed on screen). Key child elements:

Tag	Purpose
<code><meta charset="UTF-8" /></code>	Sets character encoding. ALWAYS include. Required for correct display of all languages and special characters.
<code><meta name="viewport" content="width=device-width, initial-scale=1.0" /></code>	Makes the page responsive and mobile-friendly. ALWAYS include.
<code><title>My Page</title></code>	Sets the browser tab title and the search engine result headline. Critical for SEO.
<code><link rel="stylesheet" href="style.css" /></code>	Links an external CSS stylesheet.
<code><link rel="icon" href="favicon.ico" /></code>	Sets the small icon displayed in the browser tab.
<code><script src="app.js" defer></script></code>	Links external JavaScript. "defer" loads it after the HTML is parsed.
<code><meta name="description" content="..." /></code>	Page description shown in search engine result snippets.

Full HTML5 Boilerplate -- Every Line Explained

Memorize this boilerplate. Every HTML file you create starts with this structure:

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8" />

    <meta name="viewport"
      content="width=device-width, initial-scale=1.0" />

    <meta name="description" content="Brief page description here." />

    <title>Page Title Here</title>

    <link rel="stylesheet" href="style.css" />

  </head>

  <body>

    <header>
      <h1>Site Name</h1>
    </header>

    <main>
      <h2>Welcome</h2>
      <p>Content goes here.</p>
    </main>

    <footer>
      <p>Copyright 2024 My Site</p>
    </footer>

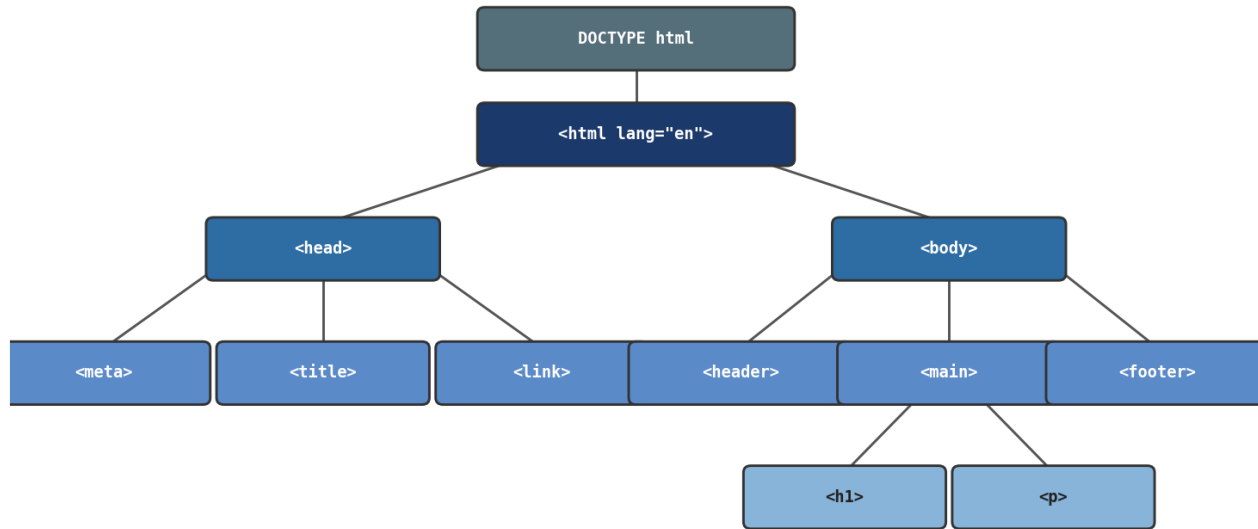
    <script src="app.js" defer></script>

  </body>

</html>
```

Document Tree Structure (DOM)

HTML Document Object Model (DOM) Tree



Each element can contain child elements -> forming a tree (parent-child hierarchy)

Figure 2.1 -- HTML Document Object Model tree showing parent-child relationships

Topic 03 -- HTML Elements, Tags and Attributes

Anatomy of an HTML Element

An HTML element typically has three parts: an opening tag (with optional attributes), content (text or child elements), and a closing tag.

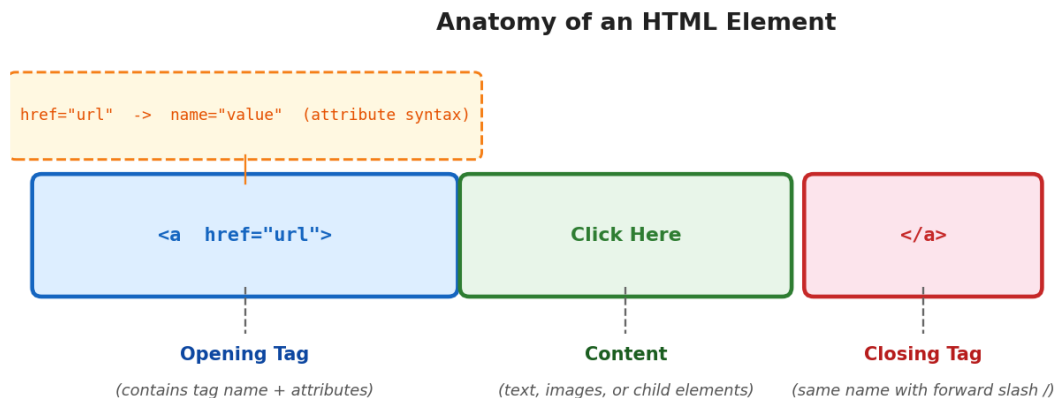


Figure 3.1 -- The three parts of an HTML element and attribute syntax

Void (Self-Closing) Elements

Void elements have no content and no closing tag. They may optionally be written with a trailing slash `/>`.

Tag	Purpose	Example
<code></code>	Embed an image	<code></code>
<code>
</code>	Line break within text	Line one Line two
<code><hr></code>	Horizontal rule (thematic divider)	<code><hr /></code>
<code><input></code>	Form input field	<code><input type="text" name="user" /></code>
<code><meta></code>	Page metadata (used in head)	<code><meta charset="UTF-8" /></code>
<code><link></code>	Link external resource (CSS, icon)	<code><link rel="stylesheet" href="style.css" /></code>

Attributes: Syntax, Rules, and Examples

Attributes provide additional information about an element. They always appear inside the opening tag and follow the format `name="value"`. Rules: always use lowercase names, always wrap values in double quotes, separate multiple attributes with spaces.


```
<a href="https://example.com" target="_blank" rel="noopener">Click Me</a>

<input type="email" name="email" placeholder="Enter your email" required />
```

Global Attributes (Available on Every HTML Element)

Attribute	Purpose	Example
id	Unique identifier -- must be unique per page	id="main-nav"
class	CSS class name(s) -- multiple elements can share	class="btn btn-primary"
style	Inline CSS (use sparingly -- prefer external CSS)	style="color: red;"
title	Tooltip text shown on mouse hover	title="Submit the form"
hidden	Hides the element from display (boolean -- no value needed)	<p hidden>Not shown</p>
lang	Language of the element content	Bonjour
data-*	Custom data attributes for storing extra data	data-user-id="42"
tabindex	Controls keyboard tab order for accessibility	tabindex="0"

Proper Nesting Rules

HTML elements must be properly nested: closing tags must appear in reverse order of their opening tags. Never overlap or interleave elements.

[CORRECT] Proper Nesting	[WRONG] Incorrect Nesting
<pre><p>Important</p> Item one Item two </pre>	<pre><p>Important</p> Item one Item two</pre>
[x] Tags closed in correct reverse order [x] li closed before /ul	[!] /p appears before /strong -- WRONG [!] li tags never closed -- INVALID

Topic 04 -- Headings

h1 Through h6 -- Visual Hierarchy

HTML provides six heading levels. h1 is the most important; h6 is the least. They create a document outline, similar to a formal paper or book structure.

HTML Heading Hierarchy -- h1 through h6



< h1 is the most important (use only once per page) | h6 is the smallest >

Figure 4.1 -- h1 through h6 visual size hierarchy

SEO Importance -- One h1 Per Page

Search engines treat headings as signals of page structure and topic. Critical rules:

- Use EXACTLY ONE h1 per page -- it is the main title, like the headline of a newspaper.
- Include your primary keyword in the h1 naturally.
- Use h2 for major sections; h3 for subsections within an h2; and so on.
- NEVER skip heading levels (e.g., do not jump from h2 directly to h4).
- Heading order must be strictly hierarchical and logical.

Accessibility -- Screen Readers and Heading Order

Screen reader users (visually impaired) navigate pages by jumping between headings using keyboard shortcuts -- like a spoken table of contents. Correct heading hierarchy is essential for WCAG (Web Content Accessibility Guidelines) compliance and is also a legal requirement in many jurisdictions.

Heading Usage Quick Reference

Heading	Use For	Example Text	Count Per Page
<h1>	Main page or article title	My Blog Post Title	Exactly ONE
<h2>	Major sections of the page	Introduction, Features, Pricing	As many as needed
<h3>	Subsections within an h2 section	Feature Details, Sub-categories	As many as needed
<h4>	Sub-subsections, detail areas	Specific option or spec	Use sparingly
<h5>	Rarely used -- very deep nesting	Fine print, nested categories	Rarely
<h6>	Almost never used in practice	Extreme edge cases only	Almost never

Topic 05 -- Paragraphs and Text Formatting

The <p> Paragraph Tag

The <p> tag defines a block of text as a paragraph. Browsers automatically add vertical spacing above and below. It is a block-level element.

```
<p>This is the first paragraph of content.</p>
<p>This is the second. Browsers add spacing between paragraphs automatically.</p>
```


 Line Break and <hr> Horizontal Rule

```
<p>123 Main Street<br />
Springfield, IL 62701</p>

<p>Section One ends here.</p>
<hr />
<p>Section Two begins here.</p>
```

Use
 only when a line break is part of the content (addresses, poems). Do NOT use
 to add spacing -- use CSS margin instead.

Text Formatting Tags -- Semantic vs Visual

HTML has semantic tags (convey meaning to browsers and screen readers) and visual tags (only change appearance). Always prefer semantic tags.

Tag	Renders As	Semantic Meaning	Use When...
	Bold	High importance, seriousness, or urgency	Content is critically important
	Bold	No semantic meaning -- purely visual bold	Stylistically bold without added importance
	<i>Italic</i>	Stress emphasis that changes meaning of sentence	Spoken emphasis alters intended meaning
<i>	<i>Italic</i>	No semantic meaning -- visual italic only	Book titles, foreign words, technical terms
<mark>	Highlighted text	Relevant or highlighted passage in context	Search result highlights, key passages
	Struck through	Deleted or removed content	Price reductions, document edits
<ins>	<u>Underlined</u>	Inserted or newly added content	Document revisions, additions
<sup>	X ² superscript	Superscript text	Math exponents, footnotes, trademarks

Tag	Renders As	Semantic Meaning	Use When...
<sub>	H ₂ O subscript	Subscript text	Chemical formulas, footnote references
<pre>	Preserves whitespace exactly	Preformatted text -- whitespace preserved	Code samples, ASCII diagrams, poems

Topic 06 -- Links (Anchor Tags)

The <a> Tag -- Syntax and Purpose

The anchor tag creates hyperlinks. The href attribute (hypertext reference) specifies the link destination.

```
<a href="https://example.com">Visit Example Site</a>
<a href="about.html">About Us</a>
<a href="#contact">Jump to Contact Section</a>
```

Absolute vs. Relative URLs

Type	Description	Example	Use When
Absolute	Full URL: protocol + domain + path	https://google.com/search	Linking to external websites
Root-relative	Path from website root (starts with /)	/about.html	Same website, any page location
Doc-relative	Path relative to the current file	../images/logo.png	Same project, local file links
Same page	Link to an element by ID on this page	#section-2	Long-page navigation anchors

target="_blank" and Security

```
<a href="https://google.com" target="_blank"
  rel="noopener noreferrer">Open Google in New Tab</a>
```

Why rel="noopener noreferrer" is required:

- **noopener:** Prevents the new page from accessing window.opener (stops potential malicious JS redirect attacks)
- **noreferrer:** Hides the referring page URL from the destination; also implies noopener

Special Link Types

Link Type	Code Example	Notes
Anchor (same page)	Contact Us	Target element needs matching id="contact"
Email (mailto)	Email Us	Opens default email client with To: pre-filled
Phone (tel)	Call Us	On mobile, opens dialer. Always include country code.

Link Type	Code Example	Notes
File download	<code>Download PDF</code>	"download" attribute triggers save dialog

Absolute vs. Relative Path Resolution Diagram

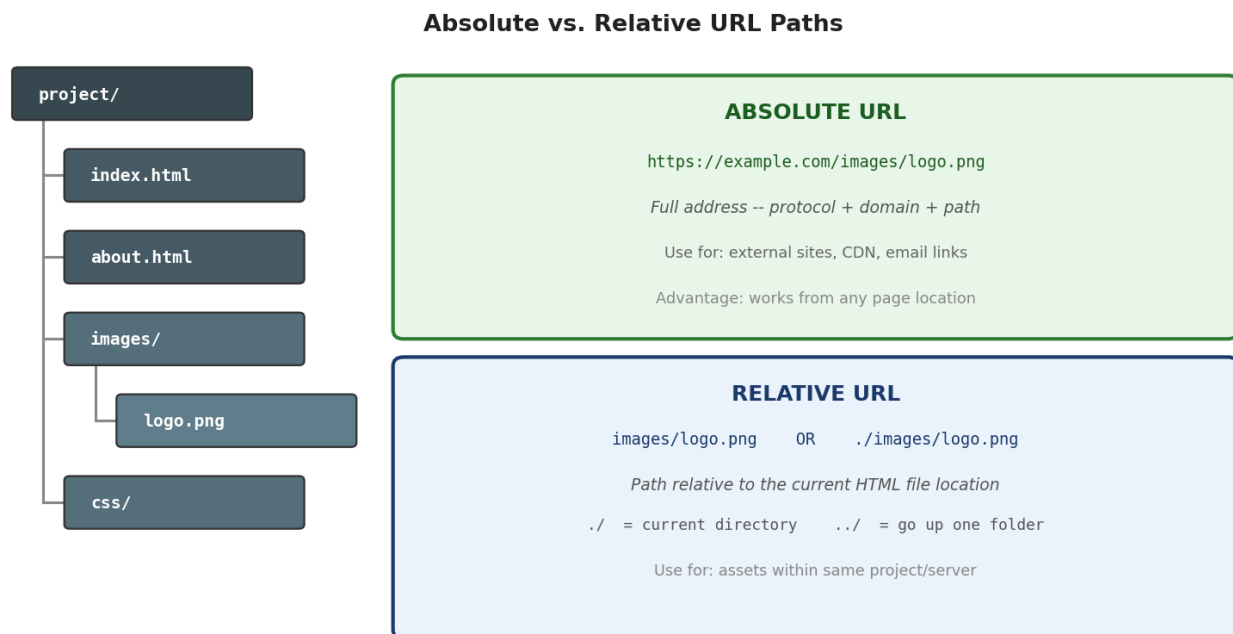


Figure 6.1 -- Absolute vs. relative URL path resolution in a project

Topic 07 -- Images

The Tag -- Required Attributes

The element embeds an image. It is a void element (no closing tag). Both src and alt are required.

```

```

Attribute	Required?	Purpose
src	YES -- required	URL or file path to the image.
alt	YES -- required	Descriptive text for screen readers and if image fails. Use empty alt="" for purely decorative images.
width	Recommended	Display width in pixels. Set both to prevent layout shift.
height	Recommended	Display height in pixels. Needed with width.
loading	Optional	loading="lazy" defers off-screen images. Great for performance.

Why Alt Text Matters

Alt text serves three critical purposes: (1) Screen readers read it aloud to visually impaired users, (2) search engines use it to index and rank images, (3) browsers display it when the image fails to load.

```

```

```





```

Image Formats Reference

Format	Best Use Case	Transparen cy	Animation	File Size
JPG / JPEG	Photos, complex real-world images	No	No	Small-Medium (lossy compression)
PNG	Logos, icons, screenshots needing transparency	Yes (alpha)	No	Medium-Large (lossless)
GIF	Simple animations, 256-color graphics	1-bit only	Yes	Small-Medium (256 color max)

Format	Best Use Case	Transparency	Animation	File Size
SVG	Icons, logos, charts (infinitely scalable)	Yes	Yes (CSS)	Very Small (vector-based)
WebP	Modern replacement for JPG+PNG -- best for web	Yes	Yes	Very Small (25-35% less than JPG)

Responsive Image Basics

```

```

```

```

```

```

Topic 08 -- Lists

Unordered List -- and

An unordered list displays items with bullet points. Use when order does not matter.

```
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Oranges</li>
</ul>
```

Bullet style controlled via CSS: `list-style-type: disc | circle | square | none`

Ordered List -- and

An ordered list shows numbered items. Use when sequence matters.

```
<ol>
  <li>Preheat the oven to 350 degrees F</li>
  <li>Mix all dry ingredients in a bowl</li>
  <li>Bake for 25 minutes until golden</li>
</ol>
```

type attribute	Numbering Style	Output Example
type="1" (default)	Arabic numerals	1, 2, 3, 4...
type="A"	Uppercase letters	A, B, C, D...
type="a"	Lowercase letters	a, b, c, d...
type="I"	Uppercase Roman numerals	I, II, III, IV...
type="i"	Lowercase Roman numerals	i, ii, iii, iv...

Definition List -- <dl>, <dt>, <dd>

A definition list pairs terms with their definitions. Ideal for glossaries, FAQs, metadata.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language -- structures web page content</dd>

  <dt>CSS</dt>
  <dd>Cascading Style Sheets -- controls visual presentation</dd>

  <dt>JavaScript</dt>
  <dd>Programming language enabling interactive web behavior</dd>
</dl>
```

Nested Lists

Lists can be nested by placing a list inside an `<li>` element:

```
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Backend
    <ul>
      <li>Node.js</li>
      <li>Python / Django</li>
    </ul>
  </li>
</ul>
```

When to Use Which List Type

List Type	Use When...	Real-World Example
<code></code>	Order does NOT matter; items have equal weight	Grocery list, features list, navigation menu items
<code></code>	Order DOES matter; sequence is important	Recipe steps, installation guide, rankings, instructions
<code><dl></code>	Term-definition pairs or name-value relationships	Glossary, FAQ section, product specifications

Topic 09 -- Block vs Inline Elements

Behavior Explained

Every HTML element has a default display behavior -- either block or inline:

BLOCK elements: Take up the full available width of their container. Always start on a new line. Stack vertically.

INLINE elements: Only take as much width as their content. Flow within surrounding text. Sit side-by-side.

Block vs. Inline Element Layout Behavior

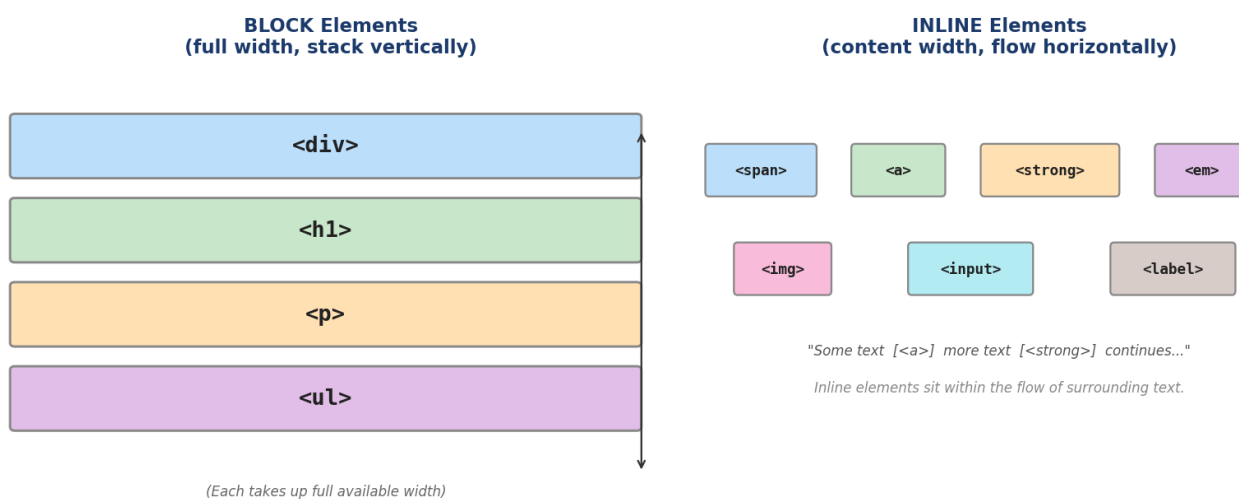


Figure 9.1 -- Block vs. inline element layout behavior comparison

Common Block-Level Elements

Tag	Purpose
<code><div></code>	Generic block container -- no semantic meaning
<code><p></code>	Paragraph of text
<code><h1></code> through <code><h6></code>	Heading levels
<code></code> , <code></code>	Unordered and ordered lists
<code></code>	List item
<code><table></code>	Data table
<code><section></code>	Semantic standalone section of content
<code><article></code>	Self-contained article or post
<code><header></code>	Page or section header

Tag	Purpose
<footer>	Page or section footer
<nav>	Navigation links
<main>	Primary content area of page (one per page)
<aside>	Sidebar or tangentially related content
<blockquote>	Extended quotation from another source
<pre>	Preformatted text block -- preserves whitespace

Common Inline Elements

Tag	Purpose
	Generic inline container -- no semantic meaning
<a>	Hyperlink anchor
	Inline image (replaced element)
	Strong importance (bold, semantic)
	Emphasized text (italic, semantic)
	Bold text (visual only)
<i>	Italic text (visual only)
 	Line break within text flow
<input>	Form input field
<label>	Label for a form element
<button>	Clickable button
<mark>	Highlighted or marked text
<code>	Inline code snippet

<div> and -- The Generic Containers

<div> and have no built-in semantic meaning. They are "blank" containers used purely for grouping and styling via CSS classes.

	<div>	
Type	Block-level	Inline
Purpose	Group block-level elements for layout or shared styling	Style or group part of text or inline elements

	<div>	
Example	<code><div class="card">...</div></code>	Price: <code>\$9.99</code>

Note: The CSS `display` property (block, inline, flex, grid, inline-block) can override any element's default display type.

Topic 10 -- HTML Comments

Comment Syntax

HTML comments are invisible to website visitors but visible in the source code. Comments do NOT affect rendering or behavior.

When TO Use Comments

- [x] Explain the purpose of a complex or non-obvious block of HTML.
- [x] Mark section boundaries in large HTML files for easy navigation.
- [x] Temporarily disable (comment out) a section of code for testing.
- [x] Leave TODO or FIXME notes for yourself or teammates.
- [x] Indicate where a component or widget begins and ends in templates.

```
<header class="site-header">

  <nav class="main-nav">
    ...
  </nav>

</header>
```

When NOT To Use Comments -- Security Warning

HTML comments are fully visible to anyone who views page source (Ctrl+U or right-click -> "View Page Source"). NEVER put sensitive information in HTML comments.

HTML is a public document. All comments are public. Never store credentials, API keys, internal server details, or any sensitive data in HTML comments.

Topic 11 -- HTML Best Practices

Apply this checklist to every HTML file you write. These rules cover correctness, accessibility, SEO, and professionalism.

Priority	Rule	Why It Matters	Code Example
[x] Required	Always declare DOCTYPE	Prevents quirks mode browser rendering	<code><!DOCTYPE html></code>
[x] Required	Set lang on	Accessibility + SEO: declares content language	<code><html lang="en"></code>
[x] Required	Set meta charset UTF-8	Correct display of all characters and symbols	<code><meta charset="UTF-8" /></code>
[x] Required	Include meta viewport	Makes site mobile responsive	<code><meta name="viewport" ... /></code>
[x] Required	Lowercase tag names	HTML5 standard and consistent style	<code><p> <div> NOT <P> <DIV></code>
[x] Required	Close all tags	Unclosed tags cause unpredictable rendering	<code><p>Text</p> NOT <p>Text</code>
[x] Required	Add alt to every image	Accessibility requirement; also needed for SEO	<code></code>
[x] Required	Only ONE h1 per page	Critical for SEO and document hierarchy	One h1 per page; multiple h2-h6 OK
[x] Best Practice	Consistent indentation (2 or 4 spaces)	Readability and team collaboration	Child elements indented from parent
[x] Best Practice	Use semantic elements	Better accessibility, SEO, and code clarity	<code><nav> NOT <div id="nav"></code>
[x] Best Practice	Quote all attribute values	Required by XHTML; consistent good practice	<code>href="page.html" NOT href=page.html</code>
[x] Best Practice	Validate at validator.w3.org	Catch errors and ensure standards compliance	https://validator.w3.org (free tool)
[!] Avoid	No sensitive data in comments	Comments are visible to everyone in source	Never: passwords, API keys, server details
[!] Avoid	Avoid inline styles	Hard to maintain; use CSS classes instead	Use class= attributes and external CSS

Validation Workflow


```
Step 1: Write your HTML and save as filename.html
Step 2: Open in browser -- check it renders correctly visually
Step 3: Open https://validator.w3.org in the browser
Step 4: Use "Validate by Direct Input" and paste your HTML
Step 5: Read through all errors and warnings carefully
Step 6: Fix each issue in your editor and re-validate
Step 7: Repeat until zero errors and zero warnings
Step 8: Repeat for every page of your project
```

End of Chapter 01 Notes -- HTML Fundamentals -- Full Stack Zero to Hero